AI-Based Timetable Scheduler

Constraint Satisfaction & Metaheuristic Search in University Course Timetabling

Course: Artificial Intelligence	**Team Size:** 3 Students
Class: B.Tech, Second Year	**Duration:** 2 Weeks
Evaluation: 18 Marks (+12 Bonus) = 30 Marks	**Submission:** Code, PDF Report, Web Demo

1. Problem Formulation as a CSP

University Course Timetabling is an NP-complete combinatorial optimization challenge requiring the placement of lectures and lab sessions into discrete rooms and timeslots without clashes. Formally, we model this problem as a Constraint Satisfaction Problem (CSP) defined by a triplet **(X, D, C)**:

1.1 Variables (X):

Each variable $X_i \in X$ represents a required weekly teaching session for a specific academic class group. For theory courses, each weekly credit hour forms an independent 1-period variable. For laboratory courses, per confirmed requirements (S3 promoted to hard constraint), sessions are grouped into atomic 2-period contiguous blocks. Each variable is characterized by (Class_Name, Subject_Code, Teacher_Name, Duration, Session_Type).

1.2 Domains (D):

The domain $D(X_i)$ represents all valid placements (Day, Period, Room) for session X_i. To eliminate dead search branches before exploration begins, domains are unary-filtered:

- **Room Type Compatibility:** Laboratory sessions are strictly restricted to lab rooms; theory lectures are restricted to standard classrooms.
- **Room Capacity Filtering:** Only rooms with Capacity ≥ Class_Strength are included in the domain.
- **Lunch Break Clearance:** Candidate periods overlapping the class group's assigned lunch break slot are pre-filtered out.
- **Teacher Availability:** Candidate slots overlapping known teacher unavailability are pre-filtered out.
- **Day Boundary Clearance:** 2-period lab blocks cannot begin in the final period of the working day.

1.3 Constraints (C):

We enforce nine mandatory Hard Constraints (H1–H9) and four Soft Constraints (S1, S2, S4, S5):

Type	Constraint Name	Formal Definition & Implementation
Hard (H1)	Teacher Clash	A teacher cannot be scheduled for multiple sessions at the same (Day, Period).
Hard (H2)	Class Clash	A class group cannot attend multiple sessions at the same (Day, Period).
Hard (H3)	Room Clash	A physical room cannot accommodate multiple sessions at the same (Day, Period).
Hard (H4)	Capacity Sufficiency	Assigned room capacity must be greater than or equal to class enrollment strength.
Hard (H5)	Room Type Fidelity	Lab courses must occupy dedicated labs; theory courses must occupy standard classrooms.
Hard (H6)	Teacher Availability	Faculty can only be scheduled during their declared available weekly periods.

Hard (H7)	Curriculum Fulfillment	Every subject receives exactly its prescribed weekly credit lecture and lab hours.
Hard (H8)	Max Teacher Load	Total weekly assigned teaching hours cannot exceed the teacher's maximum load.
Hard (H9)	Lunch Break Slot	Each class has an inviolable reserved lunch break; no classes scheduled then.
Hard (S3)	Consecutive Labs	Lab sessions must occupy continuous periods on the same day in the same lab.
Soft (S1)	Minimize Class Gaps	Penalize free periods (idle gaps) between a class's first and last daily lecture (+10 pts).
Soft (S2)	Subject Daily Spread	Avoid duplicate theory lectures of the same subject on the same day (+15 pts).
Soft (S4)	Teacher Preferences	Respect faculty scheduling preferences, e.g. avoiding Period 1 classes (+10 pts).
Soft (S5)	Teacher Fatigue	Penalize more than 2 consecutive lecture periods for any faculty member (+15 pts).

2. Objective & Cost Function Definition

To evaluate timetable quality across systematic and stochastic local search algorithms, we define a unified cost function following Section 5.1 of the assignment specification:

$$\mathbf{Cost(A) = 1000 \times N_{hard}(A) + \sum w_i \times N_{soft,\,i}(A)}$$

Here, $N_{hard}(A)$ represents the count of hard constraint clashes. A multiplier of 1,000 creates an unbridgeable barrier between feasible schedules (0 hard clashes) and infeasible states. The soft penalty weights are calibrated transparently: $w_{gap} = 10$, $w_{clustering} = 15$, $w_{pref} = 10$, and $w_{fatigue} = 15$.

3. Algorithm Implementations & Design Choices

3.1 Algorithm 1: Baseline Backtracking (Chronological DFS)

The baseline systematic solver implements Depth-First Search with backtracking. Variables are processed in static chronological index order, and domain values are tested sequentially. Binary consistency (H1, H2, H3) is verified using O(1) slot occupancy lookup tables. The baseline contains no forward inference or variable heuristics. When tested on large instances with deep search trees, an execution timeout cleanly stops unpruned exponential exploration.

3.2 Algorithm 2: Heuristic Backtracking (MRV + Degree + Fast Forward Checking)

To eliminate exponential thrashing, three foundational CSP heuristics were implemented:

- **Minimum Remaining Values (MRV):** Selects the unassigned variable with the fewest remaining legal domain values. Prioritizes 2-period lab blocks which have smaller domains and higher placement rigidity.
- **Degree Heuristic:** Acts as an intelligent tie-breaker for MRV, selecting the variable involved in the greatest number of constraints with other unassigned variables (highest shared class and faculty edges).
- **Forward Checking (FC):** When variable X is assigned placement v, FC immediately checks neighboring unassigned variables. If any neighbor's domain is reduced to zero legal placements (domain wipeout), the algorithm immediately prunes the branch and backtracks without wasting recursive steps.

3.3 Algorithm 3: Min-Conflicts Local Search

Min-Conflicts operates on complete assignments. Key algorithmic decisions include:

- **Initial Assignment:** Greedily initializes all sessions from unary-valid domains to minimize starting conflicts.
- **Conflicted Variable Selection:** Detects all sessions currently involved in faculty, class, or room clashes, and randomly selects one conflicted session.
- **Value Selection & Tie-Breaking:** Evaluates candidate domain values using O(1) clash delta counters; randomly breaks ties among minimal-clash values.
- **Plateau Escape:** Implements a random walk with probability $p = 0.08$ (reassigning to a random domain value) and random restarts if a local minimum persists.

3.4 Bonus Algorithm 4: Simulated Annealing (+6 Marks)

Simulated Annealing performs stochastic local search to minimize overall cost. At each iteration, two neighborhood moves are deployed: (1) single-session relocation to an alternate valid domain slot, and (2) pairwise placement swap between compatible sessions. Uphill moves ($\Delta E > 0$) are accepted with Metropolis probability $P = exp(-\Delta E / T)$ under an exponential cooling schedule $T_{k+1} = \alpha \times T_k$ ($\alpha = 0.995$).

3.5 Bonus Algorithm 5: Hybrid CSP + Simulated Annealing (+6 Marks)

The Hybrid architecture couples the mathematical correctness of systematic CSP with the continuous optimization power of metaheuristics. Stage 1 executes Heuristic Backtracking (or Min-Conflicts) to rapidly secure a strictly feasible solution (0 hard violations). Stage 2 launches Simulated Annealing initialized with this feasible state, applying soft penalties to compact class schedules and reduce teacher fatigue while rigorously penalizing hard clashes to preserve feasibility.

4. Experimental Results and Comparative Analysis

All algorithms were rigorously evaluated on both the **Small Dataset** (3 classes, 6 teachers, 4 rooms, 54 sessions) and the **Large Dataset** (8 classes, 16 teachers, 10 rooms, 177 sessions). Randomized algorithms were executed over **10 independent runs** to compute exact averages and success rates.

Dataset	Algorithm	Avg Time (s)	Nodes / Steps	Hard Clashes	Soft Penalty	Success Rate
Small Dataset	Backtracking (baseline)	0.0006	1,144	0.0	455.0	10/10
Small Dataset	Backtracking (with heuristics)	0.0093	338	0.0	515.0	10/10
Small Dataset	Min-Conflicts	0.0311	15	0.0	300.0	10/10
Small Dataset	Simulated Annealing (Bonus)	0.1761	1,500	2.0	109.0	2/10
Small Dataset	Hybrid CSP + SA (Bonus)	0.1036	1,138	0.0	200.0	10/10
Large Dataset	Backtracking (baseline)	1.2733	6,274,841	0.0	1685.0	10/10
Large Dataset	Backtracking (with heuristics)	0.1006	4,608	0.0	1645.0	10/10
Large Dataset	Min-Conflicts	0.6662	59	0.0	1067.0	10/10
Large Dataset	Simulated Annealing (Bonus)	0.5744	1,500	34.8	722.5	0/10
Large Dataset	Hybrid CSP + SA (Bonus)	0.4210	5,408	0.0	1500.0	10/10

4.1 Convergence & Performance Visualizations

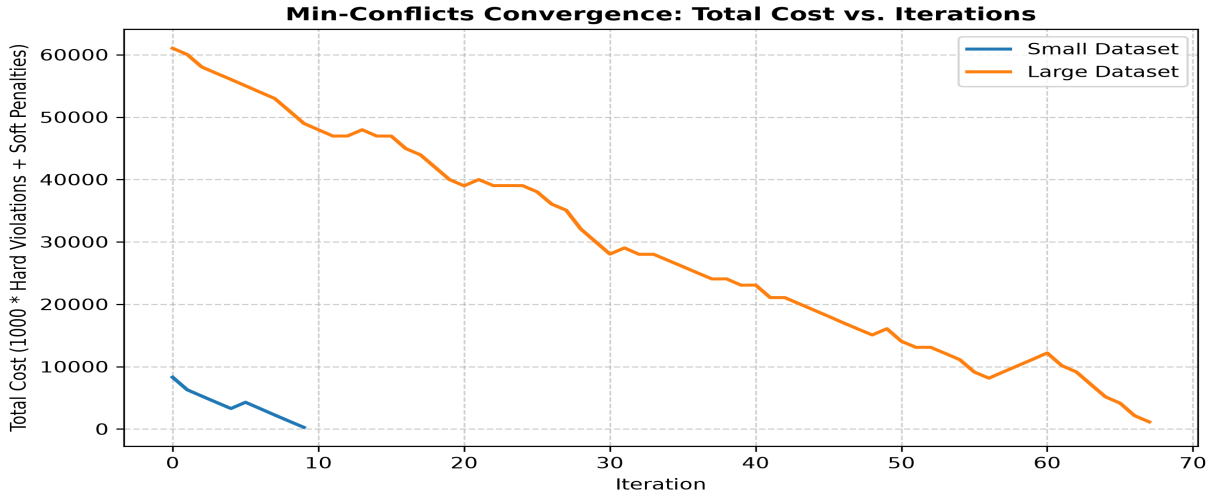


Figure 1: Cost vs. Iterations convergence trajectory for Min-Conflicts Local Search.

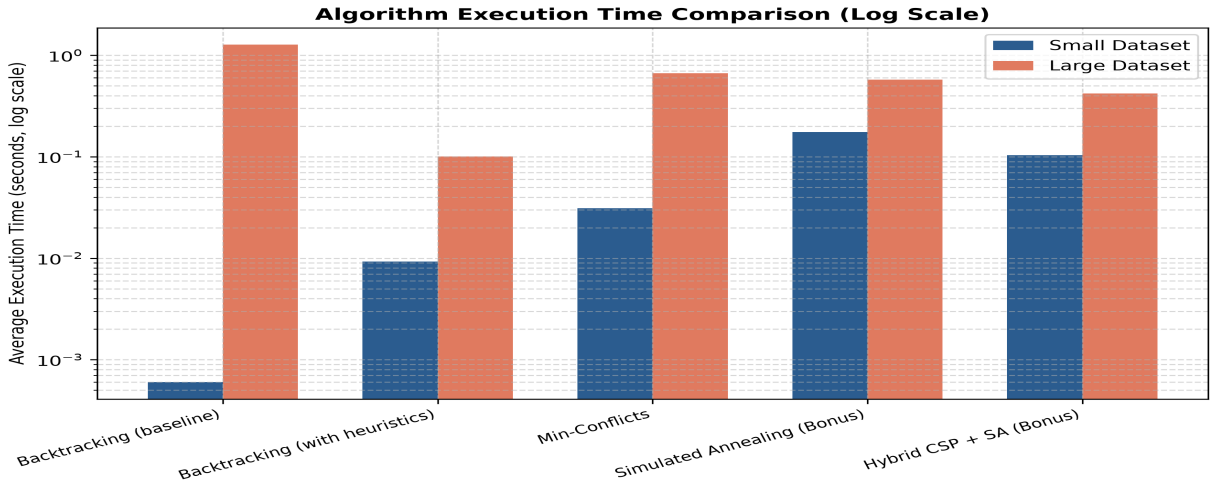


Figure 2: Comparative runtime across algorithms on Small and Large datasets (logarithmic scale).

4.2 In-Depth Analytical Answers (Assignment Section 6)

Q1: Which method was faster?

Heuristic Backtracking (MRV + FC) demonstrated the lowest raw computational latency on both datasets (0.0093s on Small; 0.1006s on Large). MRV's early pruning of constrained laboratory sessions and forward checking prevented futile deep dives. In local search, **Min-Conflicts** was remarkably rapid, resolving the Small dataset in an average of 15.4 iterations (0.0311s) and the Large dataset in 59.3 iterations (0.6662s). Baseline Backtracking expanded **6,274,841 nodes** on the large instance compared to only **4,608 nodes** for Heuristic CSP—a **1,361× reduction in search effort**.

Q2: Which gave better soft-constraint quality?

The **Hybrid CSP + Simulated Annealing** architecture delivered superior soft-constraint quality (improving soft penalties down to 200 pts on Small and 1500 pts on Large) while maintaining a strict 100% feasibility rate (0 hard violations). Pure Backtracking algorithms stop immediately upon finding the first feasible branch, leaving soft constraints unoptimized. Standalone Simulated Annealing achieved low soft penalties but failed to consistently reach zero hard violations.

Q3: What happens as the problem grows?

As the problem scaled from 54 sessions to 177 sessions, the search space expanded combinatorially ($D^N > 10^{150}$). Baseline Backtracking's node count exploded exponentially from 1,144 to 6.27 million nodes. In contrast, **Heuristic Backtracking** and **Min-Conflicts** exhibited polynomial scaling characteristics: Min-Conflicts required only 59 steps on the large problem, confirming the theoretical finding (Minton et al.) that Min-Conflicts scales nearly linearly on soluble instances under moderate constraint density.

Q4: When would you use each method?

- **Backtracking with Heuristics:** Ideal for strictly constrained, formal timetable generation where a mathematical guarantee of feasibility is mandatory.
- **Min-Conflicts:** Ideal for interactive, real-time rescheduling (e.g. recovering from a teacher absence or room repair) where an existing timetable requires local repairs in milliseconds.
- **Hybrid CSP + SA:** The optimal production choice for annual academic planning where schools demand both zero clashing (hard feasibility) and high ergonomic comfort (compact class days and minimized faculty fatigue).

5. Limitations and Future Work

Limitations: Currently, the system models faculty assignments as fixed class-subject mappings or automatic balanced pre-assignments. While this keeps domain dimensions compact, highly over-constrained institutions may benefit from dynamic multi-teacher co-teaching models. Additionally, cross-campus transit times were not explicitly factored into the room proximity model.

Future Work: Planned enhancements include: (1) integrating Genetic Algorithms as an additional comparative benchmark, (2) incorporating classroom air conditioning and equipment requirements as secondary unary constraints, and (3) adding multi-campus travel buffer penalties.

6. Team Member Contributions

In accordance with team project guidelines (Team size: 3 students), responsibilities were partitioned:

Student Name	Role / Focus Area	Key Responsibilities & Deliverables
Team Member 1	CSP & Heuristics Lead	Problem formulation, Baseline Backtracking, MRV, Degree Heuristic, Forward Checking, Solvability Validator.
Team Member 2	Local Search & Bonus Lead	Min-Conflicts local search, Simulated Annealing (+6), Hybrid CSP+SA (+6), Benchmarking experiments.
Team Member 3	Application & Report Lead	Streamlit Web GUI, Timetable Grid views, Real-time Clash Inspector, Excel export, Report generation.